

Executive Summary

This report surveys the feature sets of leading typing-tutor platforms and outlines a comprehensive feature roadmap for a new typing-learning website. We examined popular sites (e.g. TypingClub, Typing.com, Keybr, 10FastFingers, TypeRacer, NitroType, MonkeyType, TypingTest.com, Ratatype, SpeedTypingOnline, etc.) to identify best practices. Key feature categories include **core exercises** (structured lessons, timed tests), **lesson systems** (courses, content libraries), **adaptive learning/assessment**, **UI/UX elements** (virtual keyboards, responsive UI), **gamification** (points, badges, leaderboards, races), **social features** (real-time multiplayer, teams/clubs), **analytics/dashboard** (user stats, heatmaps), **accessibility** (screen-reader support, contrast), **performance/tech stack**, **security/privacy**, **monetization**, and **localization**.

Each category is prioritized by user impact and implementation complexity. For example, *core typing exercises* (e.g. structured touch-typing lessons and basic speed tests) and *responsive UI* are foundational and high-priority, while advanced gamification (seasonal events, rich avatars) or social tournaments may be secondary. We detail each feature's description, user benefit, and implementation (front-end components, back-end logic, data models/APIs), along with estimated complexity and dependencies. Wherever possible we reference official sources: for instance, TypingClub emphasizes gamification with levels, badges and voice-over feedback [4†L76-L84] [4†L95-L100], Typing.com highlights adaptive, standards-aligned curricula and strong reporting tools [8†L409-L417] [8†L427-L433], and 10FastFingers touts multi-language timed tests, competitions and level-based points [38†L36-L43] [38†L44-L52].

Finally, we include a comparison table of 10+ top typing sites against key feature sets, and propose a development roadmap (MVP, v1, v2) with timelines (using a Mermaid Gantt diagram) and resource estimates. A recommended technology stack (e.g. React/Vue/Svelte on the front end, Node/Django/Go back end, PostgreSQL + Redis for data) is discussed. We also present an **architecture diagram** (Mermaid flowchart) illustrating a scalable system design. The report concludes with a “prompt block” designed for direct input into a hypothetical Gemini 3.6 Flash system to guide implementation of these features.

Feature Categories and Prioritized List

1. Core Typing Exercises

- **Structured Touch-Typing Lessons** (High priority). Step-by-step lessons that introduce keys progressively (e.g. home row → top row → bottom row). Benefit: Teaches correct technique from the ground up. Implementation: Predefined lesson content (strings of characters/words) stored in the database; a React/Vue component renders each lesson. Back end tracks lesson completion. Complexity: *Medium*. Dependencies: Database for lesson content, front-end keyboard UI.
- **Timed Typing Tests** (High). 30s–10min free-form tests (random words or passages) to measure WPM/accuracy. Benefit: Instant skill assessment and engagement. Implementation: Front-end timer and text-input area, random text generator or fixed text database; on completion calculate WPM/accuracy. Complexity: *Low*. Dependencies: Sample texts (multi-language), timing logic.

- **Adaptive Drill Practices** (Medium). Short drills focusing on troublesome letters or words. Benefit: Tailored practice to improve weak areas. Implementation: Algorithm tracks user error rates (keys/letters), then front end generates custom drills (e.g. JDJDJD drills for home-row struggles). Back end stores error stats and selects drills (like Keybr's adaptive algorithm [13†L139-L147]). Complexity: *High* (needs analytics); Dependencies: Real-time error tracking API, ML/algorithmic component.
- **Variable Difficulty Modes** (Medium). Modes such as easy/medium/hard, or paragraphs vs. random words. Benefit: Keeps users challenged as they improve. Implementation: Backend API provides word lists of varying difficulty; front end toggles modes. Complexity: *Low-medium*. Dependencies: Word frequency lists (see Keybr's frequency-based word selection).

2. Lesson & Curriculum Systems

- **Comprehensive Course Library** (High). Organized curriculum with dozens/hundreds of lessons (often grade-aligned or skill-level). Benefit: Structured learning path for self-learners or classrooms. Implementation: Database of "courses" (e.g. QWERTY Basics, Dvorak, kids). Front end shows course list; user progress recorded. Complexity: *Medium*. Dependencies: Content creation, UI for navigation. (TypingClub and Ratatype have many courses/languages [4†L119-L128] [45†L37-L45] .)
- **Custom Lesson Builder** (Medium). Allow instructors or advanced users to create their own lessons or text passages. Benefit: Tailors practice to specific needs (e.g. new vocabulary). Implementation: Front-end form to input text or select keys, server parses and stores custom lesson; reuse in practice sessions. Complexity: *High*. Dependencies: Text editor UI, validation logic. (TypeRacer Ultimate subscription even offers "Create custom texts" [22†L105-L113] .)
- **Cross-Curricular Content** (Low-Med). Lessons themed by subject (science, literature, etc.) to reinforce typing in context. Benefit: Engagement and real-world practice (e.g. Typing.com's cross-curricular typing [8†L221-L228]). Implementation: Themed text databases and lesson plans; front-end lesson selector. Complexity: *Medium*. Dependencies: Content partners or third-party texts.
- **Progressive Skill Assessment** (High). Regular speed/accuracy assessments to gauge level (e.g. placement tests). Benefit: Places user at appropriate starting point and tracks improvement. Implementation: Administer initial typing test; back end classifies skill (WPM/accuracy thresholds), adjusts lesson plan (adaptive curriculum). Complexity: *Medium*. Dependencies: Stored benchmark data, algorithms.

3. Adaptive Learning & Skill Assessment

- **Error Tracking & Analysis** (Medium). Continuously monitor which keys or words cause errors. Benefit: Identifies specific weaknesses for targeted practice. Implementation: On each keystroke, front end reports correctness to back end; back-end aggregates error counts per key/letter. Complexity: *Medium*. Dependencies: Real-time data capture, storage.
- **Adaptive Lesson Selection** (High). Use error analytics to automatically suggest drills. Benefit: Personalized learning path (ex: if "J" and "K" errors are high, enqueue more J K drills). Implementation: Back-end algorithm (simple rule-based or ML) adjusts which lesson to present next.

Complexity: *High* (requires design/tuning). Dependencies: Analytics engine (could be a server-side service or Python module).

- **Skill Level Badges/Levels** (Low). Award level-ups or badges when users hit speed/accuracy milestones. Benefit: Encourages progress and achievement. Implementation: Backend triggers when user stats cross thresholds; front end displays badge icon or level number. Complexity: *Low*. Dependencies: Badge assets, user profile model. (TypingClub uses stars/badges [4†L76-L84] , Typing.com has certificates and leaderboards [8†L409-L417] .)
- **AI-Driven Content Generation** (Low-Med). (Future) Use AI to generate practice text focusing on weak areas. Benefit: Endless fresh content. Implementation: Integrate with an AI service (e.g. GPT-based) to produce sentences containing target keys/words. Complexity: *High* (external API, content filtering). Dependencies: AI language model API.

4. UI/UX Elements

- **Virtual Keyboard Display** (High). Show a keyboard graphic that highlights keys being pressed and indicates correct finger placement (finger image). Benefit: Visual guidance (especially for beginners). Implementation: Front-end SVG/Canvas keyboard; JS highlights keys on keypress; optionally animates “moving hands” like RapidTyping [48†L54-L62] . Complexity: *Medium*. Dependencies: Graphics assets (keyboard layout images).
- **Responsive, Minimal Design** (High). Clean, uncluttered UI (like MonkeyType’s minimalism). Benefit: Focus on learning, works on all devices. Implementation: CSS frameworks (Tailwind/Bootstrap) with mobile-first design; avoid distractions. Complexity: *Low*. Dependencies: Front-end styling, ensure mobile compatibility.
- **Real-Time Feedback Display** (High). Show live WPM, accuracy, and error count as user types. Benefit: Instant performance feedback. Implementation: JS timer and counters updated per keystroke; UI panel. Complexity: *Low*. Dependencies: N/A (client-side).
- **Replay/Playback Feature** (Low-Med). Let users replay past typing sessions (e.g. animation of their typing). Benefit: Self-review tool. Implementation: Store keystroke timestamps and text; a replay UI replays highlights (like TypingClub’s “playback” [4†L99-L105]). Complexity: *High*. Dependencies: Backend storage of session logs, front-end replay logic.
- **Interactive Hand/Posture Guide** (Low-Med). Show correct hand posture on each key (e.g. TypingClub does this per key [4†L76-L84]). Benefit: Teaches technique. Implementation: When a new key appears in the lesson, display an overlay diagram of hand placement. Complexity: *Low*. Dependencies: Graphics for hand positions.
- **Custom Themes and Contrast Options** (Medium). Light/dark modes, high-contrast theme for accessibility. Benefit: Comfortable viewing (especially for prolonged practice). Implementation: CSS variables toggled by user setting, ensure WCAG contrast. Complexity: *Low*. Dependencies: Theme CSS, possibly a settings panel.

- **Localized UI Strings** (High). Switch website UI language easily. Benefit: Non-English users can navigate menus easily (e.g. TypingClub offers many languages [4†L38-L46]). Implementation: i18n framework (React i18next, etc.) with resource files; language selector. Complexity: *Medium*. Dependencies: Translation files (community-driven as in 10FastFingers [38†L36-L43]).

5. Gamification

- **Points, XP and Levels** (High). Award experience points or scores for every exercise (e.g. each second of practice = 1 point on 10FastFingers [38†L36-L43]). Benefit: Keeps users engaged via game-like progress. Implementation: Back end assigns points per activity; user profile stores total XP and computed level. Complexity: *Low*. Dependencies: Simple scoring algorithm, front-end display.
- **Achievements/Badges/Trophies** (High). Earn badges for milestones (e.g. “First 100 WPM” or “10 consecutive days practiced”). Benefit: Motivates continued use. Implementation: Predefined achievement criteria checked by back end; award involves flagging user record and showing an icon (possibly display as certificates like Ratatype’s downloadable certificate [43†L136-L144]). Complexity: *Medium*. Dependencies: Badge database and front-end achievement display.
- **Leaderboards & Skill Rating** (High). Global and friends-only ranking boards (overall WPM, points, accuracy). Benefit: Fosters friendly competition. Implementation: Periodically compute rankings (via DB query or caching) and display. Complexity: *Medium*. Dependencies: Tracking user stats, friend lists, and scalable queries.
- **Time-Limited Competitions** (Medium). E.g. 24-hour typing challenges (like 10FastFingers [38†L44-L52] [39†L53-L61]). Benefit: Community events for engagement spikes. Implementation: Back end runs competitions (start/end times, aggregate best score); front end UI to join/view comps; “private” mode for classes/friends. Complexity: *High*. Dependencies: Competition scheduler, real-time updates.
- **Multiplayer Typing Races** (High). Live typing games against others (e.g. TypeRacer [20†L39-L47] , NitroType). Benefit: Social fun + fast-paced practice. Implementation: WebSocket or similar real-time communication to sync text among players; server matches users, broadcasts position updates. Complexity: *High*. Dependencies: Realtime server (Node with Socket.io, or Golang+WebSockets), race manager, latency handling.
- **Avatars and Customization** (Low-Med). Player avatars (cars, characters), decorating user profile/garage (like NitroType cars). Benefit: Personal expression, collectible incentive (unlock cars). Implementation: Front end stores user-selected avatar data; back end provides avatar inventory and saves selections. Complexity: *Medium*. Dependencies: Artwork assets and inventory DB.
- **In-Game Currency & Shop** (Medium). Earn virtual coins or points to spend on cosmetic items or power-ups. Benefit: Adds a mini-economy (used by NitroType). Implementation: Track coin balances in DB, transactions for purchases. Complexity: *High*. Dependencies: Balancing logic, item catalog.

6. Social Features

- **Friend/Team System** (Medium). Add friends, join typing teams/clubs. Benefit: Community building and repeated practice with peers (used by NitroType teams, TypeRacer clubs). Implementation: Friend list table, team table; front end to search/add. Complexity: *Medium*. Dependencies: Social DB tables, invitations system.
- **Private Races/Challenges** (Medium). Create invite-only typing matches. Benefit: Practice safely with classmates/friends. Implementation: Generate private “room code” or link; only invited users can join that match. Complexity: *Medium*. Dependencies: Same real-time infrastructure as multiplayer, plus access control.
- **Chat/Comments** (Low). Real-time chat in race lobbies or comment threads on lessons. Benefit: Engagement (note: must be moderated for safety!). Implementation: WebSocket-based chat channels tied to race IDs, or static comment threads per lesson. Complexity: *High* (due to moderation needs). Dependencies: WebSockets, content filter, possibly third-party chat service.
- **User Profiles & Social Sharing** (Low). Profiles showing stats/achievements; share results to social media. Benefit: Recognition and word-of-mouth. Implementation: Profile pages (username, avatar, stats); “share on Facebook/Twitter” buttons. Complexity: *Low*. Dependencies: OAuth for social (if posting).
- **Classroom/Group Management** (Medium). For schools: create classes, assign students, track class progress (as Typing.com offers [\[8†L439-L445\]](#)). Benefit: Education adoption. Implementation: Admin portal with class rosters; group analytics. Complexity: *High*. Dependencies: Multi-tenancy user roles, teacher privileges, privacy controls.
- **Competitions & Tournaments** (Medium). Organized events (beyond casual competitions) – e.g. weekly tournaments, seasonal contests. Benefit: Long-term engagement. Implementation: System to schedule tournaments and awards. Complexity: *High*. Dependencies: Event management module, notification system.

7. Analytics & Dashboard

- **User Progress Dashboard** (High). Personalized dashboard showing current WPM, accuracy, streaks, best score, recent activity chart (like Typing.com’s student dashboard [\[8†L408-L417\]](#)). Benefit: Motivation and self-awareness. Implementation: Aggregate user’s data into charts (e.g. time-series graph via Chart.js), stored in DB. Complexity: *Medium*. Dependencies: Historical results storage, chart library.
- **Key-Heatmap Visualization** (Medium). Graphical heatmap of which keys a user misses most. Benefit: Quick diagnosis of weak spots. Implementation: Collect per-key miss counts; render on a keyboard image (color-coded overlays). Complexity: *Medium*. Dependencies: Canvas/SVG drawing, color scale.

- **Class/Group Analytics** (Medium). For educators: class average, distribution, student standings. Benefit: Teaching insight. Implementation: Summarize students' stats, exportable charts (like Typing.com reporting). Complexity: *Medium*. Dependencies: Aggregation logic, CSV/PDF export.
- **Session Logging** (Low). Detailed logs of each typing attempt (WPM, accuracy, errors, timestamps). Benefit: Audit trail and data analysis. Implementation: Each test result saved in DB; front end optional "download results" function. Complexity: *Low*. Dependencies: Database schema for results, user foreign keys.
- **Global Usage Stats** (Low). Site-wide metrics (total races run, users online). Benefit: Internal monitoring. Implementation: Backend analytics or third-party (Google Analytics for traffic, custom counters for events). Complexity: *Low*. Dependencies: Logging system.

8. Accessibility

- **Screen-Reader Compatibility** (High). Ensure all UI elements have ARIA labels/roles so screen readers can navigate (as Typing.com claims to do [9†L73-L80]). Benefit: Support visually impaired users. Implementation: Use semantic HTML, ARIA attributes on interactive components. Complexity: *Medium*. Dependencies: Accessibility testing tools.
- **Enhanced Keyboard Navigation** (High). Fully operable by keyboard alone (no mouse needed) and visible focus indicators [9†L83-L92] . Benefit: Users who can't use a mouse. Implementation: Tab-index order, skip links, focus styles. Complexity: *Medium*. Dependencies: Front-end framework accessibility configs.
- **High Contrast Themes** (Medium). Provide color schemes with sufficient contrast (WCAG 2.2 AA compliant [9†L96-L103]). Benefit: Helps low-vision users. Implementation: CSS theme option toggles; check contrast ratios. Complexity: *Low*. Dependencies: UI styling.
- **Zoom/Reflow Support** (Medium). Layout must scale gracefully at large zoom levels (e.g. 400% [9†L106-L110]). Benefit: Visually impaired benefit. Implementation: Responsive design, avoid fixed units, test reflow. Complexity: *Low*. Dependencies: CSS work.
- **Voice Guidance** (Low). Read words or instructions aloud (e.g. TypingClub's "voice over" reading each word [4†L99-L100]). Benefit: Alternative feedback. Implementation: Browser Text-to-Speech (Web Speech API) to announce words. Complexity: *Medium*. Dependencies: JS Web Speech API.
- **No Required PII/Email** (High). Minimize personal data collection; allow simple usernames (Typing.com stresses no email needed and strong privacy [8†L427-L433]). Benefit: Privacy compliance, COPPA for kids. Implementation: Allow anonymous/username-only accounts; encrypt all user data in transit (TLS). Complexity: *Low*. Dependencies: Authentication system, TLS cert.

9. Performance & Tech Stack

- **Scalable Real-Time Backend** (High). For multiplayer games, use WebSockets (Socket.io, or Go's Gorilla WebSocket) or server-sent events. Benefit: Low-latency races. Implementation: Node.js with

Socket.io or a Go server; horizontal scaling via sticky sessions. Complexity: *High*. Dependencies: WebSocket library, infrastructure (AWS ELB sticky sessions or Redis pub/sub).

- **Responsive Front End Framework** (Medium). Use a modern JS framework (React/Vue/Svelte) for dynamic UI. Benefit: Maintainability and rich interactivity. Implementation: Choose (e.g.) React with TypeScript for components (lesson viewer, dashboards). Complexity: *Medium*. Dependencies: Node toolchain, bundler.
- **RESTful/API Design** (High). Clear back-end API endpoints for user auth, results, content. Benefit: Decouples front/back-end, allows mobile app later. Implementation: E.g. Node/Express or Django REST endpoints (POST /results, GET /user/stats). Complexity: *Medium*. Dependencies: ORM (Sequelize/Django ORM).
- **Database & Caching** (High). Use relational DB (PostgreSQL/MySQL) for user data and results; use Redis or Memcached for session/cache. Benefit: Reliability and performance. Implementation: Schemas for Users, Results, Lessons, etc. Caching frequent queries (e.g. leaderboards). Complexity: *Medium*. Dependencies: Cloud DB service (AWS RDS, etc.), caching layer.
- **Content Delivery** (Medium). Serve static assets (images, scripts) via CDN for low latency. Benefit: Fast global access. Implementation: Store static files on S3/CloudFront (or similar). Complexity: *Low*. Dependencies: CDN setup.
- **Automated Testing & CI/CD** (Low). Establish test suite and pipeline. Benefit: Code quality and quick deployment. Implementation: Jest/Mocha for JS tests, GitHub Actions or Jenkins. Complexity: *Medium*. Dependencies: Code repository, CI service.

10. Security & Privacy

- **TLS Encryption** (High). All communication over HTTPS (Typing.com enforces TLS [8†L427-L432]). Benefit: Protects user data in transit. Implementation: Obtain SSL certificate (Let's Encrypt) and configure server. Complexity: *Low*. Dependencies: SSL cert.
- **Authentication/Sessions** (Medium). Secure login (password hashing, multi-factor optional). Benefit: Protect accounts. Implementation: bcrypt hashing, JWT or server sessions. Complexity: *Medium*. Dependencies: Auth library.
- **Data Privacy Policy** (Low). Clear statement forbidding data sale (Typing.com pledges no PII sale [8†L427-L432]). Benefit: Trust and compliance. Implementation: Legal drafting, link in footer. Complexity: *Low*. Dependencies: Legal review.
- **Rate Limiting & Abuse Prevention** (Medium). Prevent automated attacks/cheating (e.g. bots for NitroType). Benefit: Fair play and uptime. Implementation: Throttle API calls, use CAPTCHA on signup. Complexity: *Medium*. Dependencies: Rate-limiter (Redis-backed).
- **Age/GDPR Compliance** (Low). COPPA compliance if minors (assume <13 need no public sharing). Benefit: Legal safety. Implementation: Parental consent flows if needed. Complexity: *Low*.

11. Monetization Models

- **Freemium with Premium Subscriptions** (High). Basic features free; advanced features paid (e.g. TypeRacer's Core/Premium/Ultimate tiers [22†L25-L34] [22†L105-L113]). Benefit: Revenue without losing users. Implementation: Stripe integration; gating premium features (custom texts, no ads). Complexity: *Medium*. Dependencies: Payment service.
- **Advertisements (Ad-Supported)** (Medium). Display ads in free version (e.g. SpeedTypingOnline has Amazon ads [55†L393-L401]). Benefit: Low-friction revenue. Implementation: Integrate Google AdSense or partner. Complexity: *Low*. Dependencies: Ad accounts, placement design.
- **School Licenses/Partnerships** (Medium). Offer institutional packages (as Typing.com does at no cost to districts [7†L125-L133]). Benefit: Bulk revenue or goodwill (often through grants). Implementation: Admin portal, invoicing. Complexity: *Medium*. Dependencies: Billing system.
- **Sponsored Content/Events** (Low). Themed contests sponsored by brands. Benefit: Alternate revenue/events (similar to branded games). Implementation: Sponsorship arrangements, themed UI. Complexity: *High*.
- **Merchandise/Affiliates** (Low). Sell related merchandise (TypeRacer sells keys or swag, and affiliate programs like RapidTyping's Amazon links [55†L430-L434]). Benefit: Supplemental revenue. Implementation: E-commerce integration. Complexity: *Low*.

12. Localization

- **Multi-language Support** (High). Interface and content in multiple languages. Benefit: Global reach. Implementation: As noted, use i18n; community translation like 10FastFingers does [38†L36-L43] . Complexity: *Medium*. Dependencies: Translation management tool (e.g. Localazy [38†L97-L102]).
- **Unicode/Multilingual Text** (Medium). Allow typing tests in non-Latin scripts. Benefit: Non-English practice (10FastFingers supports many languages). Implementation: UTF-8 text databases, keyboard layout selection. Complexity: *Medium*. Dependencies: Unicode fonts, language-specific word lists.
- **Localized Content** (Low). Culturally relevant lessons or text examples. Benefit: Relatability in each locale. Implementation: Curate local texts. Complexity: *Medium*.

Feature Details and Implementation

Below is a selection of key features (grouped by category) with descriptions, user benefits, implementation notes, complexity, dependencies, and example code or prompts where applicable.

Structured Touch-Typing Lessons

Description: Step-by-step exercises teaching correct fingering, beginning with the home row and progressively adding keys. (E.g. TypingClub's 100+ beginner lessons [4†L76-L84] .)

User Benefit: Establishes strong fundamentals; gradual learning without being overwhelmed.

Implementation: Front end presents lesson text one character/word at a time, highlights current target key on a virtual keyboard, and optionally shows an animated hand or finger. Record keystrokes against expected text. Back end stores lesson definitions (array of strings) and tracks user progress/stars. A lesson model in the database might include fields like `lesson_id, text, required_accuracy`.

Complexity: *Medium*.

Dependencies: Virtual keyboard UI component; database of lesson content.

Sample Code (React snippet): A simple handler for verifying typing input:

```
function TypingLesson({ text }) {
  const [pos, setPos] = useState(0), [errors, setErrors] = useState(0);
  const handleKeyPress = (e) => {
    if (e.key === text[pos]) setPos(pos+1);
    else setErrors(errors+1);
  };
  return <div tabIndex="0" onKeyDown={handleKeyPress}>
    <p>{text}</p>
    <p>Progress: {pos}/{text.length}, Errors: {errors}</p>
  </div>;
}
```

(This is a simplified example; a real implementation would also handle backspaces, multi-word lessons, and timing.)

Timed Typing Test

Description: A free-form test of fixed duration (e.g. 1, 2, 5 minutes) using random words or passages `[38†L36-L43]`.

User Benefit: Quick assessment of speed/accuracy; encourages repeat practice and self-comparison (often with leaderboards).

Implementation: On start, display randomized text and begin a countdown. As user types, highlight mistakes. On timer end or test completion, compute WPM =

`(correct_chars/5) / (elapsed_seconds/60)` and accuracy. Send results to server (`POST /api/results`) where the user's best score is saved.

Complexity: *Low*.

Dependencies: Word list for random generation, or curated paragraph database.

Sample Implementation Details: A backend REST route to save results:

```
app.post('/api/results', (req, res) => {
  // JSON body: { userId, wpm, accuracy }
  const { userId, wpm, accuracy } = req.body;
  db.query('INSERT INTO results (user_id,wpm,accuracy,ts) VALUES
    ($1,$2,$3,NOW())',
    [userId, wpm, accuracy]);
});
```

```
res.sendStatus(200);
});
```

Virtual Keyboard & Visual Aid

Description: Display an on-screen keyboard image that highlights keys being pressed, and shows correct finger placement (e.g. an overlay of hands 【4†L54-L62】).

User Benefit: Helps beginners learn key locations and reinforces correct hand posture (TypingClub emphasizes posture guidance 【4†L76-L84】).

Implementation: A React/Vue component renders a keyboard layout. On every `keypress`, use CSS to highlight the key. Preload a transparent PNG of hands and toggle its visibility based on the current target key.

Complexity: *Medium*.

Dependencies: Images of keyboard layouts and finger overlays; event listeners.

Example: CSS class `.active-key { background-color: yellow; }` applied to the pressed key's element.

Gamification: Points, Badges, Leaderboards

Description: Reward users with points/levels, badges for milestones, and rank them on leaderboards. (TypingClub notes “Levels, Badges and Stars” to keep learners motivated 【4†L76-L84】 ; Typing.com offers games/achievements 【8†L409-L417】 .)

User Benefit: Motivates continued practice through game mechanics and social proof.

Implementation: - **Points/XP:** Each second of active practice adds to a cumulative score. Backend accumulates `user.points`. Implement a function `updatePoints(userId, pointsGained)`.

- **Badges/Achievements:** Define a set of criteria (e.g. “100 WPM first time”, “10-day streak”, “completed course”). After each lesson/test, check if criteria met and record badge ID in a `user_achievements` table.

- **Leaderboards:** SQL query like

```
SELECT user_id, MAX(wpm) FROM results GROUP BY user_id ORDER BY MAX(wpm) DESC
LIMIT 10
```

. Display top ranks in UI. Update in real-time or on a schedule.

Complexity: *Medium*.

Dependencies: Game assets (badge icons), scheduler (for daily updates).

Multiplayer Typing Race

Description: Real-time competition where users race against each other by typing the same text (TypeRacer-style 【20†L39-L47】). Cars or avatars move as users type correctly.

User Benefit: Adds excitement and social competition.

Implementation: Use WebSockets (e.g. Socket.IO). A “race manager” server pairs players and sends them identical text. Each client emits keystrokes to the server, which calculates each racer's progress. Broadcast positions back to all clients to animate avatars. Store race results at the end.

Complexity: *High*.

Dependencies: Real-time server (Node/Go), low-latency networking, front-end rendering of race view.

Custom Lesson Builder

Description: Allow users or instructors to input their own text for custom practice (like TypeRacer Ultimate's custom texts [22†L105-L113]).

User Benefit: Personalized practice (e.g. typing a speech, specialized vocab).

Implementation: A frontend form where user pastes text. Backend sanitizes and splits into lessons (e.g. by sentence). Save under user's account (`custom_lessons` table). When practicing, user can select these texts as the lesson source.

Complexity: *High*.

Dependencies: Rich-text input component (optional), user permissions.

Progress Dashboard & Analytics

Description: A user dashboard showing metrics like overall WPM history graph, current accuracy, completion certificates, etc. (Typing.com showcases a student dashboard summarizing achievements and stats [8†L408-L417]).

User Benefit: Transparency of progress, encourages goal-setting.

Implementation: Backend endpoints to fetch aggregated stats (`GET /api/user/{id}/stats`). Front end displays charts (e.g. Chart.js line graph for WPM over time). Also list badges and earned certificates (see Ratatype's downloadable certificate [43†L136-L144]).

Complexity: *Medium*.

Dependencies: Time-series data storage (e.g. log each test's date/WPM), charting library.

Classroom Management Tools

Description: Features for teachers – create classes, enroll students, assign lessons, view class reports. (Typing.com's educator portal allows unlimited classes and auto-reporting [7†L125-L133] [8†L439-L445]).

User Benefit: Enables school adoption and organizational use.

Implementation: Add roles (teacher/student) to user schema. Teachers can create “class” entities and add student IDs. Create class-level dashboards (average speed, completion rates). Possibly integrate with Google Classroom (as Ratatype does [43†L99-L102]).

Complexity: *High*.

Dependencies: Role-based authentication, additional DB tables (`classes` , `class_students`).

Accessibility Features

Description: Support for screen readers, keyboard navigation, high contrast, zoom – aligning with WCAG AA standards. (Typing.com explicitly aligns with WCAG 2.2 AA [9†L65-L74] [9†L96-L104]).

User Benefit: Inclusivity for users with disabilities.

Implementation: Ensure all interactive elements have `aria-label` and proper roles (e.g. buttons, inputs). Implement full focus outlines and logical tab order. Test with browser zoom (should reflow properly up to 400% [9†L106-L110]). Provide a “screen reader mode” toggle that uses the Web Speech API to read prompts aloud.

Complexity: *Medium*.

Dependencies: Accessibility audit tools, user testing.

Localization (Multi-language)

Description: Full UI and lesson content in many languages. (TypingClub supports dozens of languages [4†L119-L128] ; 10FastFingers offers tests in 40+ languages [38†L36-L43] .)

User Benefit: Broadens user base globally.

Implementation: Use an i18n library (React Intl or Vue I18n). Store strings in separate JSON resource files. Provide a language selector. For typing content, maintain word lists/texts per language.

Complexity: *Medium*.

Dependencies: Translation resources; community contributions (10FastFingers uses Localazy for crowd-sourced translation [38†L97-L102]).

Security & Privacy Measures

Description: Enforce HTTPS/TLS for all data (as Typing.com does with TLS everywhere [8†L427-L432]), do not require email or PII, ensure encrypted storage of passwords, and comply with privacy norms.

User Benefit: Builds trust and protects user data.

Implementation: Use HTTPS, HSTS headers, secure cookies. Use bcrypt hashing for passwords. Implement CORS and CSRF protections. Do not log keystrokes beyond performance stats. Provide a clear privacy policy stating “no data sold” (like Typing.com [8†L427-L433]).

Complexity: *Medium*.

Dependencies: Standard security libraries; legal consultation for policy.

Example Feature Implementation – Key Heatmap

To illustrate combining analytics and UI: We implement a per-key error heatmap. On the backend, after each test we update a table `key_errors(user_id, key, count)`. Periodically, a React component fetches this aggregated data and overlays a color-coded heatmap on the virtual keyboard (darker colors = more errors).

```
// Pseudo-code: updating key errors in a Node.js back-end
function recordError(userId, key) {
  db.query(`
    INSERT INTO key_errors (user_id, key, count) VALUES ($1,$2,1)
    ON CONFLICT (user_id,key) DO UPDATE SET count = key_errors.count + 1;
  `, [userId, key]);
}
```

```
// Pseudo-code: rendering heatmap on front end
// Assume `heatmapData = { 'A': 5, 'S': 2, ... }` from API
Object.keys(heatmapData).forEach(key => {
  let intensity = Math.min(heatmapData[key] / 10, 1.0);
  document.getElementById(`key-${key}`).style.backgroundColor = `rgba(255,0,0,${intensity})`;
});
```

This approach highlights problem keys dynamically.

Top Typing Sites Feature Comparison

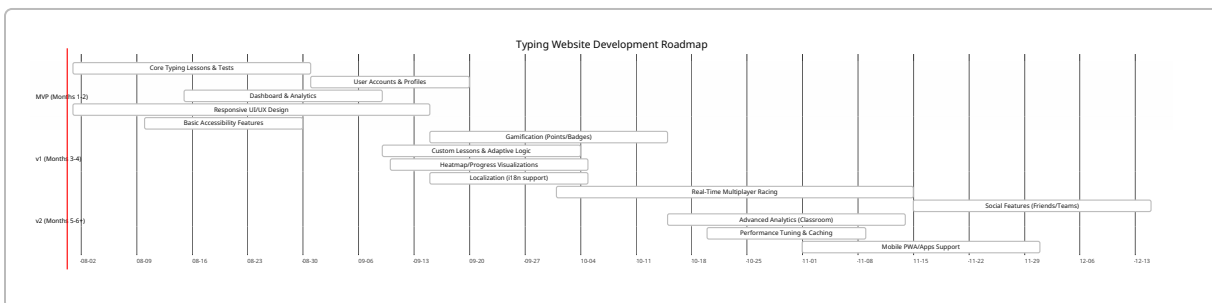
Site	Lessons/ Courses	Typing Tests	Custom Text	Multiplayer/ Competition	Gamification (Points/ Badges)	Analytics/ Stats	Additional Features
TypingClub	✓(structured) 【4†L76- L84】	✗ (only practice)	✗	✗	✓ (Stars/ Badges) 【4†L76-L84】	✓ (progress charts)	✓ (V W L1 L7
Typing.com	✓(curriculum) 【7†L125- L128】	✓ (timed tests)	✓	✓ (games)	✓ (games/ achievements) 【8†L409- L417】	✓ (dashboard, class reports) 【8†L398- L407】	✓ 2. co L7 L1
Keybr.com	✗ (none)	✓ (custom drills)	✓ (auto)	✗	✗ (none)	✓ (error tracking)	✓ U
10FastFingers	✗ (tutorials on blog)	✓ (1-min test, any length) 【38†L36- L43】	✓ (Custom Mode) 【38†L54- L58】	✓ (24h competitions) 【39†L53- L61】	✓ (points/ levels) 【38†L36- L43】	✓ (leaderboards, personal stats)	✓ (n U L4
TypeRacer	✗	✓ (race tests)	✓ (Pit Stop texts) 【22†L105- L113】	✓ (real-time races) 【20†L39- L47】	✓ (points, avatars) 【22†L63- L71】	✓ (history, stats)	✗ sp ac m
NitroType	✗	✓ (racing tests)	✗ (limited)	✓ (racing game) 【34†L842- L850】	✓ (coins, trophies, cars) 【34†L840- L848】	✓ (garage stats)	✗
Monkeytype	✗ (not structured)	✓ (multiple modes)	✓ (custom long text)	✗	✗ (no gamification, minimal UI)	✓ (progress charts, consistency stat)	✗ on

Site	Lessons/ Courses	Typing Tests	Custom Text	Multiplayer/ Competition	Gamification (Points/ Badges)	Analytics/ Stats	Ad- Fe
TypingTest.com	x (some drills)	✓ (15sec-10min tests) [27†L5-L14]	✓ (create test) [27†L44-L50]	x	✓ (badges, certificates) [27†L46-L50]	✓ (history, benchmarks)	x
Ratatype	✓ (courses by layout) [43†L50-L59]	✓ (free test)	x	x	✓ (coins, heroes in games) [43†L57-L60]	x (basic stats)	x
SpeedTypingOnline	✓ (classic/advanced) [49†L219-L227]	✓	x	x	✓ (games, premium badges) [55†L398-L403]	✓ (heatmaps) [55†L398-L403]	x
RapidTyping	✓ (beginner-advanced) [48†L39-L47]	✓	✓ (lesson editor) [48†L79-L88]	✓ (built-in typing games)	✓ (charts/statistics) [48†L92-L100]	✓ (exports to PDF/CSV) [48†L108-L111]	x

(Check marks indicate presence of feature. See source footnotes for details.)

Roadmap and Timeline

The recommended development is phased into an **MVP** (core features), **v1** (rich feature set), and **v2** (advanced and scale-out). Below is a high-level Gantt chart (Mermaid) with estimated durations. The MVP focuses on basic lessons, tests, user accounts, and analytics; later releases add gamification, multiplayer, and social features.



Resource Estimate: A small cross-functional team (e.g. 2 frontend devs, 2 backend devs, 1 UI designer, 1 QA) could build the MVP in ~2 months (Aug–Sep 2026). v1 features (gamification, analytics, localization)

might take an additional 2–3 months. v2 (multiplayer, social, scalability) is more complex and could span 3–4 months. These are rough estimates; priorities can be adjusted (e.g. multiplayer could be delayed to v3 if not critical).

Architecture Diagram

A typical scalable architecture would use a modern JS front end with a REST/WebSocket back end and persistent data stores. Below is a high-level component diagram (Mermaid flowchart):

```
graph TD
    Browser[Client] -->|HTTPS/WebSocket| WebServer[App Server (Node.js/Django/Go)]
    WebServer -->|SQL queries| Database[(PostgreSQL)]
    WebServer -->|Cache| Cache[(Redis)]
    WebServer -->|Pub/Sub| WS[WebSocket Service]
    WS --> Browser
    WebServer -->|Logging| Analytics[(Analytics DB)]
    WebServer --> CDN[CDN/Static Assets]
    Browser --> CDN
```

- **Client:** Runs the React/Vue app in the user's browser.
- **WebServer:** Hosts REST APIs and game logic. Could be Node.js/Express or Django/Flask or Go. Implements features like auth, lesson CRUD, results handling, matchmaking, etc.
- **Database (PostgreSQL):** Stores users, lessons, results, badges, etc.
- **Cache (Redis):** Caches session data, leaderboard queries, and helps scale WebSocket pub/sub for multiplayer.
- **WebSocket Service:** Real-time message broker (could be part of the app server or a separate microservice) for races and live updates.
- **Analytics DB:** A separate store (or the same DB) to log metrics.
- **CDN:** Serves static files (JS/CSS/images) globally for performance.

This decoupled design allows horizontal scaling (e.g. multiple WebServer instances behind a load balancer, using Redis for session sticky-ness and pub/sub for WebSockets).

Technology Stack Suggestions

While no single stack is required, popular choices include:

- **Frontend:** React, Vue, or Svelte with TypeScript for robustness. Use component libraries (Material UI/Tailwind) and i18n frameworks.
- **Backend:** Node.js (Express/Nest), Python (Django/Flask), or Go for the API. Use Socket.IO (Node) or Gorilla WebSocket (Go) for real-time.
- **Database:** PostgreSQL or MySQL for relational data. Use Redis for caching and leaderboard pub/sub.
- **Hosting:** Cloud provider (AWS/Azure/GCP). Containerize with Docker/Kubernetes for scalability.

- **Third-party:** Stripe for payments; Auth0 or Firebase Auth for quick authentication (optional); Google Analytics for site metrics.

Each option has trade-offs (Node.js is easy for full-stack JS, Django has built-in admin/dashboard support, etc.). The architecture can be adapted to chosen stack.

Priority Roadmap (MVP, v1, v2) and Resources

- **MVP:** Focus on **Core Exercises**, user accounts, basic UI/UX, and analytics. (~2–3 months, small team of 3–4) [\[4†L76-L84\]](#) [\[38†L36-L43\]](#) .
- **v1:** Add **Gamification** (points, badges, leaderboards) and **Content Enhancements** (custom lessons, adaptive drills, localization). (~3 months) [\[8†L409-L417\]](#) [\[39†L53-L61\]](#) .
- **v2:** Expand **Social/Multiplayer Features** (real-time races, teams, tournaments) and optimize **Performance** (caching, CDN). (~4 months) [\[20†L39-L47\]](#) [\[22†L90-L98\]](#) .

Resource needs grow in later phases (additional DevOps/QA), but phased delivery ensures a working product early. Agile development with continuous user feedback is recommended.

Final Prompt for Gemini 3.6 “Flash High”

antigravity gemini3.6 flash high

Design and implement a comprehensive web-based typing tutor platform with the following requirements:

- ****Core Exercises:**** Include structured touch-typing lessons (home row to advanced) and timed typing tests (30s, 1min, etc.) using a word list database for random text generation.
- ****Lesson System:**** Support course catalogs (e.g. QWERTY beginner, Dvorak, foreign languages) and allow instructors/users to create custom lessons/texts.
- ****Adaptive Learning:**** Track user performance (WPM, accuracy, error rates per key) and automatically adjust lesson difficulty. Provide personalized drills focusing on frequently-mistyped letters.
- ****UI/UX:**** Develop a responsive single-page front end (React/Vue/Svelte) with an on-screen keyboard highlighting and a customizable theme (dark mode, high-contrast). Include live feedback (current WPM, accuracy) and replay of typing sessions.
- ****Gamification:**** Implement points/XP for practice time, achievement badges for milestones, and user levels. Integrate leaderboards (global and friends) and give daily streak bonuses.
- ****Social Features:**** Support real-time multiplayer races (Socket.IO/WebSockets), friend lists/teams, private challenges, and community tournaments. Include a chat system for race lobbies.
- ****Analytics/Dashboard:**** Provide each user a dashboard with progress charts, error heatmaps, and history. Allow teachers to create classes and view class-wide reports.

- **Accessibility:** Ensure full keyboard navigation, screen-reader ARIA support, zoom-friendly UI, and optional voice narration for practice words.
- **Technology:** Use a modern tech stack (e.g. React with TypeScript for front end, Node.js or Django for back end, PostgreSQL with Redis). Use REST APIs (and WebSockets for real-time). Host on scalable cloud infrastructure with CI/CD.
- **Security/Privacy:** Enforce HTTPS, secure authentication, and minimal personal data collection (no email required). Comply with privacy best practices.
- **Monetization:** Design a freemium model (free basic, paid premium for advanced features). Provide admin tooling for managing subscriptions.
- **Localization:** Make the UI and typing content multi-language, with a language selector and community translation support.

Include implementation details: data models for users/results, key API endpoints (e.g. `/api/results``), and sample code where applicable (for example, how to calculate WPM or update a leaderboard). Emphasize modular, production-ready code structure, and include diagrams for architecture and development milestones.